OHS REMOTE — CAPSTONE DOCUMENTATION

Capstone347 • CSD-235

# Project Management

GitHub Projects + Milestones Workflow

Capstone347 — CSD-235

April 14, 2026

# Contents


**1 Overview** **2**

**2 Tooling** **2**
- 2.1 GitHub Projects . . . 2
- 2.2 Milestones . . . 2

**3 Workflow** **3**
- 3.1 Columns . . . 3
- 3.2 Issue lifecycle . . . 3
- 3.3 How responsibilities were distributed . . . 3

**4 Why this approach worked for a small team** **4**

**5 What we would do differently** **4**

**6 Evidence** **4**
- 6.1 Board view . . . 5
- 6.2 Milestones view . . . 6
- 6.3 Issue-to-PR linking . . . 7

**7 Links** **7**

# 1 Overview

This document describes how the Capstone347 team planned, tracked, and delivered the OHS Remote project. It covers the tools we chose, how work was distributed, how issues flowed from creation to closure, and why this approach fit a small student team working across two repositories (backend and frontend).

The short version: we ran the project on a single GitHub Projects board, used Milestones to group sprint-sized chunks of work, and tied every code change to an issue through pull requests. No third-party project management tool was introduced.

# 2 Tooling

## 2.1 GitHub Projects

Our board lives at:

**Note**

`https://github.com/orgs/Capstone347/projects/2`

We chose GitHub Projects for three reasons:

1. **Zero onboarding cost.** Every team member already had a GitHub account and was already pushing code to the organization. Adding a second tool (Jira, Trello, Linear, Asana) would have meant another login, another notification stream, and another context switch.
2. **Native PR and issue linking.** Because the board is part of GitHub, an issue automatically tracks the pull requests that reference it. Moving a PR through review implicitly moves the underlying issue through its workflow columns.
3. **One source of truth.** Code, issues, milestones, and the board all live in the same organization (Capstone347). The same URL that shows a merged PR also shows the issue it closed and the milestone it belonged to.

## 2.2 Milestones

We used GitHub Milestones to define sprint-sized chunks of work. Each milestone represented a cohesive deliverable — for example, "Order creation flow", "Stripe payment integration", "SJP generation pipeline". Issues were filed against a milestone so that, at any moment, the team could see which work items were required to close out the current milestone and which belonged to a later phase.

Milestones gave us two things the board alone could not:

- **A closeable unit of progress.** When every issue under a milestone is closed, the milestone closes. That is a concrete, visible "we finished X" moment for the team and the instructor.
- **A way to defer scope without losing it.** Ideas that were clearly out of scope for the current phase were filed against a later milestone rather than discarded or left as TODO comments in the code.

# 3 Workflow

## 3.1 Columns

The board used three workflow columns:

- **Todo** — the issue has been described, assigned to a team member, and is waiting to be worked on.
- **On Review** — the issue has a pull request open against it and is waiting for a teammate to review, test, and approve the change.
- **Closed** — the pull request has been merged and the issue has been closed.

We deliberately kept the column count small. A three-column board is fast to read, fast to update, and does not reward procrastination in a "In Progress" stage.

## 3.2 Issue lifecycle

A typical work item flowed through the project like this:

1. **Issue created.** A team member (or the team in a planning session) writes a GitHub issue describing the work. The issue is assigned to the person who will do the work and tagged with the milestone it belongs to.
2. **Assignee picks up the issue.** The issue sits in *Todo* until the assignee is ready to start. When work begins, they create a branch named after the issue (e.g., `47-sjp-job-status-endpoint`).
3. **Pull request opened.** When the branch is ready, the assignee opens a pull request. The PR description references the issue with `Closes #47` so GitHub links them automatically. The issue moves from *Todo* to *On Review*.
4. **Peer review.** Another team member reviews the PR, pulls the branch, and tests the change locally. They leave inline comments or request changes. The author iterates until the reviewer approves.
5. **Merge and close.** When the PR is approved and CI is green, the reviewer (or the author, by agreement) merges the PR. GitHub automatically closes the referenced issue, which moves the card from *On Review* to *Closed*.

This lifecycle meant that the board columns were never out of date on purpose. Moving a card required a real code action (opening a PR, merging a PR), not a manual click.

## 3.3 How responsibilities were distributed

Work was divided roughly by subsystem rather than by horizontal layer. For example, one team member owned the Stripe checkout and webhook integration end-to-end; another owned the SJP generation pipeline and its OpenAI integration; another owned the admin dashboard and order management. This avoided the merge-conflict churn that comes from two people editing the same files at the same time.

When a cross-cutting concern came up — for example, a shared Pydantic schema change or a migration that touched multiple tables — the responsible party opened a PR and explicitly tagged the other affected owners as reviewers before merging.

## 4 Why this approach worked for a small team

- **Low ceremony, high visibility.** A student team does not need a five-column Kanban board, story points, or burndown charts. We needed to know who was doing what, what was waiting for review, and what was done. Three columns and a Milestones view gave us exactly that.
- **PR-driven state changes.** Because column transitions were tied to PR actions, nobody had to remember to update the board. The board reflected reality because it was downstream of the actual code actions.
- **Milestones as deadlines.** Each milestone had a due date aligned with the course schedule. Open issues under a milestone whose due date was approaching became the team's focus automatically, without needing separate "priority" tags.
- **No extra tool to learn.** Every team member was already comfortable with GitHub from previous coursework. Adding a second tool would have increased cognitive load without adding information.

## 5 What we would do differently

For completeness, a few honest notes on where this workflow was imperfect:

- We sometimes opened PRs without first creating an issue, which meant the board missed a card for a piece of work that was actually shipped. A stricter "issue first, branch second" rule would have closed that gap.
- Milestone due dates drifted in the final two weeks as features took longer than estimated. For a future project we would build in a buffer milestone explicitly labeled as "slack" rather than silently extending the active one.
- We did not formally distinguish bug reports from feature work on the board. Labels would have made it easier to see, at a glance, how much of the remaining work was new functionality versus fixing something we had already shipped.

None of these hurt the final deliverable, but all three are small improvements a follow-up team could adopt.

## 6 Evidence

## 6.1 Board view

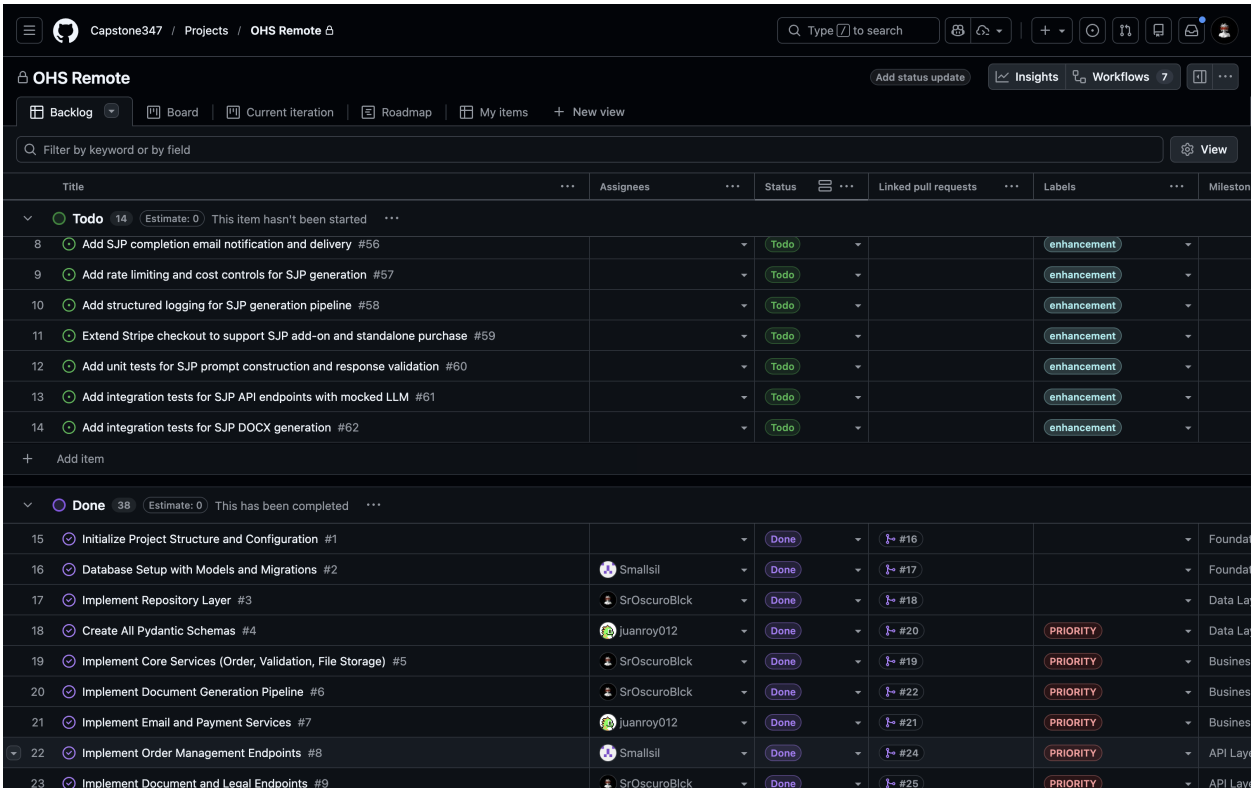


Figure 1: GitHub Projects board showing the three workflow columns (Todo, On Review, Closed) with issues distributed across the team.

## 6.2 Milestones view

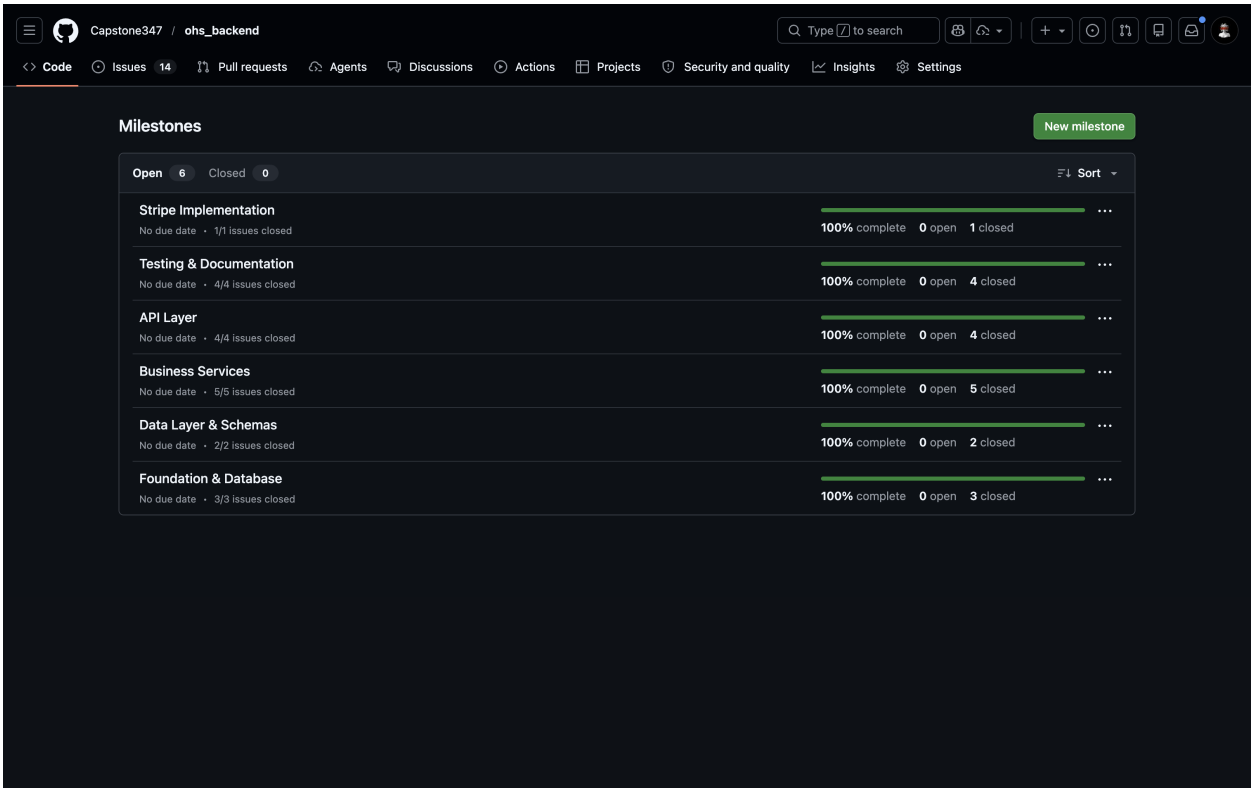


Figure 2: GitHub Milestones view grouping issues into sprint-sized deliverables, each with its own progress bar and due date.

## 6.3 Issue-to-PR linking

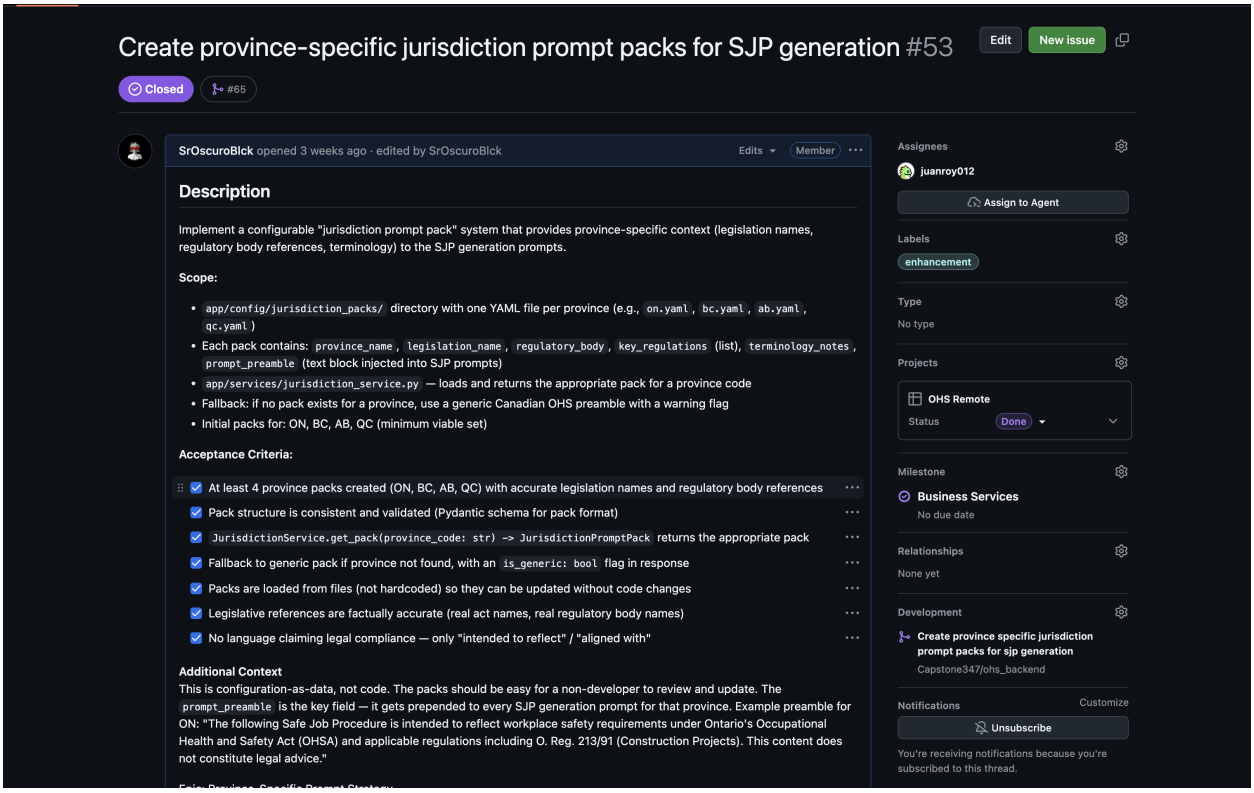


Figure 3: Example issue showing the linked pull request created by the assignee, the review thread, and the merge that automatically closed the issue.

# 7 Links

- **Project board:** https://github.com/orgs/Capstone347/projects/2
- **Backend repository:** https://github.com/Capstone347/ohs_backend
- **Frontend repository:** https://github.com/Capstone347/ohs_frontend
- **Organization:** https://github.com/Capstone347